

“Design and Development of a Smart Hostel Management System using Node.js, Express and MySQL”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

By

ANANYA :2520090011

B. SRIVALLI :2520090220

SHANMUKHI :2520090076

Under the Esteemed Guidance of

[Name of the Guide]

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Design and Development of a Smart Hostel Management System using Node.js, Express and MySQL”** is a record of Bonafide work of **ANANYA - 2520090011, B. SRIVALLI - 2520090220, SHANMUKHI - 2520090076** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

ANANYA – 2520090011
B. SRIVALLI – 2520090220
SHANMUKHI – 2520090076

K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING
&
DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is certify that the mini project based report entitled **“Design and Development of a Smart Hostel Management System using Node.js, Express and MySQL”** is a bonafide work done and submitted by **ANANYA - 2520090011, B. SRIVALLI - 2520090220, SHANMUKHI - 2520090076** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**. The work was carried out as part of the course Database Systems Engineering and Distributed Backend Development (25CS1302E), Section 13, Team 2.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish is to express our sincere thanks to all those who have assisted us in one way or the other for the completion of our project.

Our greatest appreciation to our Course Coordinator and our guide **[Name of the Guide]**, Department of Computer Science, which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)/Dr. Kaja Shareef (CSIT)**, Head of the Department for Computer Science Engineering/Department for Computer Science and Information Technology for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted us directly or indirectly for successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

ANANYA – 2520090011

B. SRIVALLI – 2520090220

SHANMUKHI – 2520090076

Abstract

Hostels are a vital part of campus life, yet in many institutions the daily work of running them – recording student details, allocating rooms, collecting fees and handling complaints – is still done on paper registers and scattered spreadsheets. This manual approach leads to data duplication, errors, slow searching and delays in updating records. To tackle this, our project focuses on the Design and Development of a Smart Hostel Management System. Our core aim was to build a centralised, database-driven web application that simplifies hostel administration and reduces manual effort.

The system is structured around interconnected modules for Student Management, Room Management, Room Allocation, Fee Management and Complaint Management, supported by Authentication and Authorization, an Admin Dashboard and Reports. Authorized users can add, view, update, delete and search records, check room availability, track allocation history, record fee payments and register and update complaints. The frontend is built with HTML, CSS and JavaScript; the backend uses Node.js with Express and exposes a REST API; and all data is held in a single, normalised MySQL database with primary and foreign keys that link students, rooms, allocations, fees and complaints.

The system applies sound database-engineering practices: normalisation up to 3NF to remove redundancy, capacity and occupancy checks to prevent room overbooking, parameterised queries to prevent SQL injection, hashed passwords, role-based access (Admin, Warden, Staff and Student) and validation on both the frontend and the backend. In summary, this project delivers a simple, secure and efficient digital solution that improves the accuracy of hostel records, speeds up information retrieval and saves administrative time, while providing a foundation for scalable backend development.

Index

S. No.	Chapters	Topics	Page.no
		Acknowledgement	
		Abstract	
1	Introduction	1.1 Background of the project 1.2 Problem statement 1.3 Scope of the project 1.4 Objective(s) 1.5 Importance of the application 1.6 Target users/audience	8-11
2	System Requirements	2.1 Hardware requirements 2.2 Software requirements 2.3 Development tools and frameworks	12-13
3	Technology Stack	3.1 Front-end (HTML/CSS/JavaScript) 3.2 Back-end (Node.js, Express.js) 3.3 Database (MySQL) 3.4 Version Control (Git, GitHub) 3.5 APIs or External services (REST API)	14-16
4	System Architecture	4.1 High-level architecture diagram 4.2 Description of each layer (frontend, backend, database) 4.3 Deployment architecture (local, cloud, CI/CD pipelines)	17-18
5	Design	5.1 ER Diagram 5.2 Database schema	19-21
6	Implementation	6.1 Module-wise implementation 6.2 Front-end logic (UI rendering, state management) 6.3 API integration 6.4 Authentication & Authorization	22-26
7	Features	7.1 List of main features 7.2 How each feature works (user perspective + technical description)	27-29
8	Testing	8.1 Postman testing (frontend/backend) 8.2 Integration testing 8.3 Tools used for testing 8.4 Test cases and results	30-32
9	Deployment	9.1 Steps to deploy 9.2 Environment configuration 9.3 Hosting of frontend and backend	33-34

S. No.	Chapters	Topics	Page.no
10	Challenges & Limitations	10.1 Issues faced during development 10.2 Solutions applied 10.3 Current limitations or known bugs	35-36
11	Future Enhancements	11.1 Planned features 11.2 Possible integrations or optimizations	37
12	Conclusion	12.1 Summary of the project 12.2 What was achieved 12.3 Skills learned during development	38-39
13	References	- Books, tutorials, APIs, documentation sites used	40
14	Appendices	- Screenshots of the app - Sample code snippets - Installation/setup instructions - User manual or guide - Geo Tag photos with guide - Review forms with guide signatures	41-47

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

A hostel is far more than a place to sleep. For a large residential university it is a small community that has to be organised every single day: students arrive and leave, rooms are allotted and changed, fees fall due, and problems such as a broken fan or a leaking tap must be reported and fixed. Behind every one of these activities sits a record, and the quality of those records decides how smoothly the hostel runs.

In many institutions these records are still maintained manually. Student details are written in registers, room charts are drawn on notice boards, fee receipts are filed in folders and complaints are noted down on loose sheets or conveyed verbally. Every piece of information lives in a different place and in a different format. When the warden needs to know which rooms are vacant, who has not paid the hostel fee, or whether a complaint has been resolved, someone has to search through several files and compare them by hand. This is slow, tiring and error-prone.

The idea for this project emerged from observing exactly these inefficiencies. The Smart Hostel Management System is a database-based web application that brings all hostel information into one centralised and structured platform. Instead of many disconnected registers, the system stores students, rooms, allocations, fees and complaints in a single relational database, and gives authorised users a simple web interface to add, view, update, delete and search that information whenever it is needed.

Modern web technology makes such a system practical and affordable. A browser-based interface built with HTML, CSS and JavaScript needs nothing to be installed on the user's computer. A Node.js and Express backend can expose the hostel's data and rules through a clean REST API, and MySQL provides a reliable relational store with keys and constraints that keep the data consistent. Together they form a three-layer architecture in which the interface, the business logic and the data are separated, which makes the system easier to maintain and to extend.

From an academic point of view, the project brings together several core areas of computer science taught in the course Database Systems Engineering and Distributed Backend Development:

1. **Database design:** identifying entities and relationships, drawing an ER diagram, defining primary and foreign keys and normalising the schema.
2. **Backend development:** building a REST API with Node.js and Express that performs CRUD operations on the database.
3. **Web technology:** building a responsive, easy-to-use frontend with HTML, CSS and JavaScript.
4. **Security:** authentication, role-based authorization, password hashing, input validation and protection against SQL injection.

5. **Software engineering practice:** modular design, version control with Git and GitHub, testing and documentation.

To summarise, the background of this project rests on three observations. First, manual hostel record keeping is slow and unreliable. Second, current technology makes a centralised, database-driven solution simple to build and run. Third, a well-designed backend and database provide a solid foundation on which more advanced services – notifications, online payments and analytics – can later be added.

1.2 Problem Statement

In a typical hostel, information about students, rooms, fees and complaints is maintained in separate paper registers or spreadsheets. Because these records are not connected to each other, the administration has no single, reliable view of what is happening in the hostel. A student's name may be written differently in the fee register and the room register; a room may be shown as vacant on one sheet and full on another; a complaint may be lost between the student who raised it and the staff member who should fix it.

The core problem addressed by this project is the lack of a centralised, structured and easily searchable system for managing hostel information. The specific difficulties can be summarised as follows:

Problem	Effect on hostel administration
Student records maintained manually	Details are scattered, difficult to update and easy to lose.
Room allocation hard to track	It is unclear who lives in which room and since when.
Room availability not updated quickly	Vacant rooms are not identified in time; rooms may be overbooked.
Fee records hard to manage	Pending and paid fees cannot be checked quickly.
Complaints hard to track systematically	Complaints are delayed, forgotten or left unresolved.
Manual records cause duplication and errors	The same data is written many times and often contradicts itself.
Searching and updating takes more time	Simple questions need long manual searches.
Lack of centralised information	Overall efficiency of the administration is reduced.

Traditional management therefore leads to data errors, delays and reduced administrative efficiency. The project aims to solve these interrelated problems through the following resolutions:

1. Replace scattered records with one centralised MySQL database.
2. Link students, rooms, allocations, fees and complaints through primary and foreign keys so that the data is always consistent.
3. Prevent room overbooking by checking capacity and occupancy before every allocation.
4. Allow quick searching and updating of records through a web interface.

5. Protect the data through login, role-based access, hashed passwords and validated input.

In summary, the problem statement identifies a clear gap between the hostel administrator's need for a reliable, unified record system and the fragmented manual methods currently in use. The proposed Smart Hostel Management System is designed to fill that gap.

1.3 Scope of the Project

The scope of the project covers the day-to-day administrative operations of a student hostel. The system is a web application accessed through a browser by authorised users, and its data is stored in a central relational database. The following areas are included in the scope:

- **Student management** – adding, viewing, updating, deleting and searching student records.
- **Room management** – maintaining room details (room number, block, floor, capacity), tracking occupancy and checking availability.
- **Room allocation** – assigning a student to a room, changing the room and keeping the allocation history.
- **Fee management** – recording fee payments and viewing payment status.
- **Complaint management** – registering complaints and updating their status until they are resolved.
- **Authentication and authorization** – login and role-based access for Admin, Warden, Staff and Student users.
- **Admin dashboard and reports** – a central page to reach all modules, with summaries of students, rooms, fees and complaints.

The following are outside the scope of the present version and are listed as future work in Chapter 11: a mobile application, online payment gateway integration, e-mail/SMS notifications, QR-based attendance, cloud deployment and microservice architecture.

1.4 Objectives of the Project

The main objective of the project is to simplify hostel administration and reduce manual effort by using a centralised database. The detailed objectives are:

1. Develop a centralised hostel management system.
2. Maintain student information in a structured database.
3. Manage hostel rooms and room availability.
4. Manage student-room allocation.
5. Maintain fee payment records.
6. Record and manage complaints.
7. Provide CRUD (Create, Read, Update, Delete) operations for authorised users.
8. Reduce data redundancy and manual effort.
9. Improve information retrieval.

10. Provide a foundation for scalable backend development.

1.5 Importance of the Application

The importance of the Smart Hostel Management System can be seen from the point of view of everyone who is connected with the hostel.

For the administration, a single source of truth replaces many separate files. The warden can see at a glance how many students are housed, how many rooms are available, how many allocations are active and how many complaints are open. Decisions can therefore be taken on accurate, up-to-date information rather than on guesswork.

For students, the system brings transparency. A student's room, fee status and complaint status are recorded in one place, so there is less confusion and fewer disputes about who paid, who was allotted which room and whether a problem has been attended to.

For the institution, centralised records improve accountability and reduce paperwork. Data entered once is reused everywhere, which removes duplication and the errors that duplication causes. The use of a normalised relational database also means that the system can grow with the number of students without becoming harder to manage.

From a learning perspective, the project gives the team practical experience of the complete lifecycle of a database-backed application – requirement analysis, ER modelling, schema design, SQL implementation, REST API development, frontend development, integration, testing and documentation.

1.6 Target Users / Audience

User	How the user benefits from the system
Administrator	Has full system access: manages all records, views reports, controls access and monitors the whole hostel.
Warden	Manages day-to-day hostel operations such as student records, room allocation and complaint follow-up.
Staff	Handles operational work such as updating room details, recording fee payments and resolving complaints assigned to them.
Student	Views personal details, room allocation, fee status and registers or tracks complaints.
Institution management	Gets accurate summaries of occupancy, fees and complaints for planning and reporting.

The interface is deliberately simple and clean so that users with little technical background can use it after a short introduction.

CHAPTER -2 SYSTEM REQUIREMENTS

2.1 Hardware Requirements

The Smart Hostel Management System is a lightweight web application. It does not need any special hardware. The tables below list the minimum and recommended configuration for the development machine, and for a computer that hosts the application in a hostel office.

Component	Minimum requirement	Recommended
Processor	Dual-core, 2.0 GHz	Quad-core i5 / Ryzen 5 or better
RAM	4 GB	8 GB or more
Storage	5 GB free disk space	20 GB free (SSD preferred)
Display	1366 × 768 resolution	1920 × 1080 resolution
Network	Local network / Wi-Fi	Stable broadband connection
Input devices	Keyboard and mouse	Keyboard and mouse

On the client side, any computer, laptop, tablet or smartphone with a modern web browser can be used to access the system, since all processing of business rules and data storage takes place on the server.

2.2 Software Requirements

Software	Purpose	Version / Note
Operating system	Platform for development and hosting	Windows 10/11, Linux or macOS
Node.js	JavaScript runtime for the backend	LTS release
npm	Package manager for Node.js libraries	Installed with Node.js
Express.js	Web framework for the REST API	Installed through npm
MySQL Server	Relational database	MySQL 8.x
MySQL Workbench / CLI	Creating tables and running queries	Any recent version
Web browser	Running the frontend	Chrome, Edge, Firefox or Safari
Postman	Testing REST API endpoints	Desktop or web version
Git	Version control	Latest stable release

2.3 Development Tools and Frameworks

The following tools and frameworks were used to design, build and manage the project:

- **Visual Studio Code** – the code editor used for writing HTML, CSS, JavaScript and Node.js code, with its integrated terminal and Git support.
- **Node.js and Express.js** – the runtime and framework for the backend REST API.

- **MySQL with MySQL Workbench** – for database design, table creation and query testing.
- **Postman** – for sending requests to the API and checking responses.
- **Git and GitHub** – for version control, team collaboration and storing the project repository.
- **Browser developer tools** – for debugging the frontend, inspecting network calls and testing responsiveness.

CHAPTER -3 TECHNOLOGY STACK

The technology stack of the Smart Hostel Management System is organised as three cooperating layers – frontend, backend and database – supported by development and version-control tools. The stack was chosen because every component is widely used, well documented, free to use and suitable for a student project that is intended to scale.

Layer	Technologies
Frontend	HTML, CSS, JavaScript
Backend	Node.js, Express.js
Database	MySQL
Communication	HTTP / REST API (JSON)
Development & version control	VS Code, Git, GitHub

3.1 Front-End Design

The frontend is the part of the system that users see and touch. It is built with the three core web technologies:

- **HTML** gives each page its structure – the navigation bar, hero section, login form, tables and forms. The project follows a multi-page structure with a home page (index.html), a login page (login.html) and separate pages for the dashboard and each module.
- **CSS** provides the visual design. A common stylesheet (css/style.css) is shared by all pages so that the fonts, colours, buttons, cards and tables look consistent. The design uses a soft purple-and-pink gradient theme, rounded cards, and a responsive layout that adapts to different screen sizes.
- **JavaScript** adds behaviour: validating form input, showing and hiding the password, sending requests to the REST API with fetch, and filling tables and dashboard counters with the data that the server returns.

The home page presents the system name, a short description (“A modern digital platform to manage students, rooms, allocations, fees and complaints in one place”), and buttons for Get Started and Explore Features. The login page uses a two-panel layout: an introduction panel on the left and a sign-in card on the right. Once the user logs in, the dashboard presents module cards and live summary counts.

3.2 Back-End Design

The backend is built with **Node.js**, a JavaScript runtime that lets the team use the same language on both the client and the server, and **Express.js**, a minimal web framework that simplifies routing, middleware and request handling.

The backend's responsibilities are to: receive HTTP requests from the frontend; validate the input; check that the user is logged in and has permission; build and run parameterised SQL queries against MySQL; and return the result as JSON with an appropriate HTTP status code. The code is organised by module (students, rooms, allocations, fees, complaints, authentication) so that each module has its own routes and logic, which keeps the project readable and easy to maintain. Commonly used npm packages in such a stack include express, mysql2 for database access, cors for cross-origin requests, dotenv for configuration and bcrypt for password hashing.

3.3 Database Design

MySQL is the relational database management system used to store all hostel data. A relational database was chosen because hostel information is naturally structured: a student has many allocations, fees and complaints; a room has many allocations. These one-to-many relationships are handled cleanly by primary and foreign keys, and the database itself can enforce rules such as “each allocation must refer to an existing student and an existing room”.

The database contains five core tables – students, rooms, allocations, fees and complaints – together with a table for user accounts and roles used by the authentication module. The schema is normalised up to the third normal form (3NF) to remove redundancy. Details are given in Chapter 5.

3.4 Version Control

Git is used to track every change made to the source code, and **GitHub** hosts the central repository. Version control allows the three team members to work on different parts of the project at the same time, to merge their work, to return to an earlier working version if something breaks, and to keep a history of how the project grew. The GitHub repository also serves as the place from which the final project is demonstrated and submitted.

3.5 APIs or External Services

The system does not depend on any paid or third-party external service. Instead it exposes its own **REST API**, through which the frontend communicates with the backend. REST (Representational State Transfer) uses the standard HTTP methods – GET to read, POST to create, PUT to update and DELETE to remove – on resource URLs such as /api/students. Data is exchanged in JSON format.

Module	Endpoints
Students	GET /api/students GET /api/students/:id POST /api/students PUT /api/students/:id DELETE /api/students/:id
Rooms	GET /api/rooms POST /api/rooms PUT /api/rooms/:id

Module	Endpoints
Allocations	POST /api/allocations GET /api/allocations PUT /api/allocations/:id
Fees	POST /api/fees GET /api/fees
Complaints	POST /api/complaints GET /api/complaints PUT /api/complaints/:id

Because the frontend talks to the backend only through this API, the same API can later be reused by a mobile application or by other services without changing the database or the business logic.

CHAPTER -4 SYSTEM ARCHITECTURE

4.1 High-Level Architecture Diagram

The Smart Hostel Management System follows a layered client–server architecture. A user (Admin, Warden, Staff or Student) works in the browser; the browser runs the HTML/CSS/JavaScript frontend; the frontend sends HTTP requests to the REST API; the Node.js and Express backend processes those requests; and the MySQL database stores the five main groups of data – students, rooms, allocations, fees and complaints.

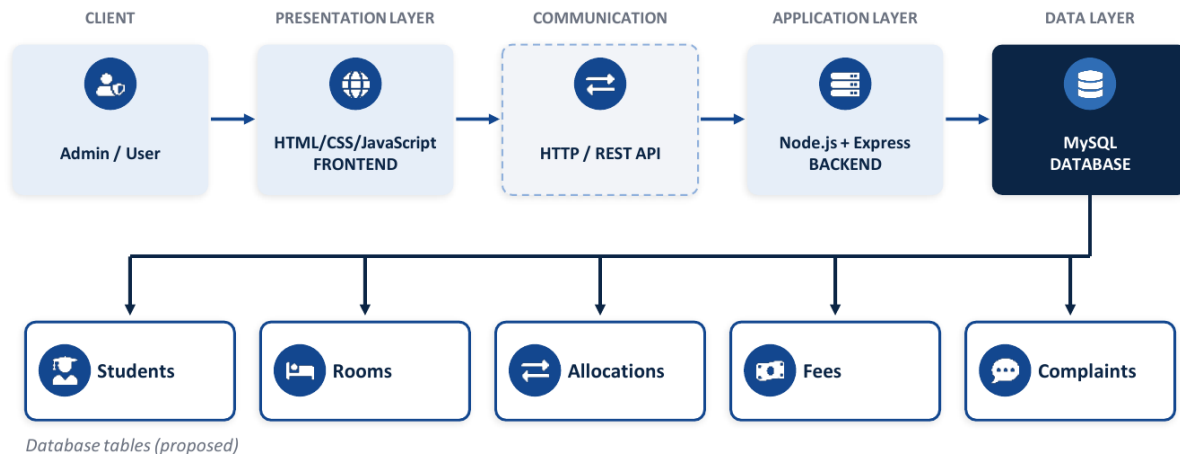


Figure 4.1 – High-level system architecture

The overall flow of a single user action through these layers is shown in Figure 4.2. Every action follows the same path – frontend, backend, database and back to the user interface – which makes the system predictable and easy to debug.

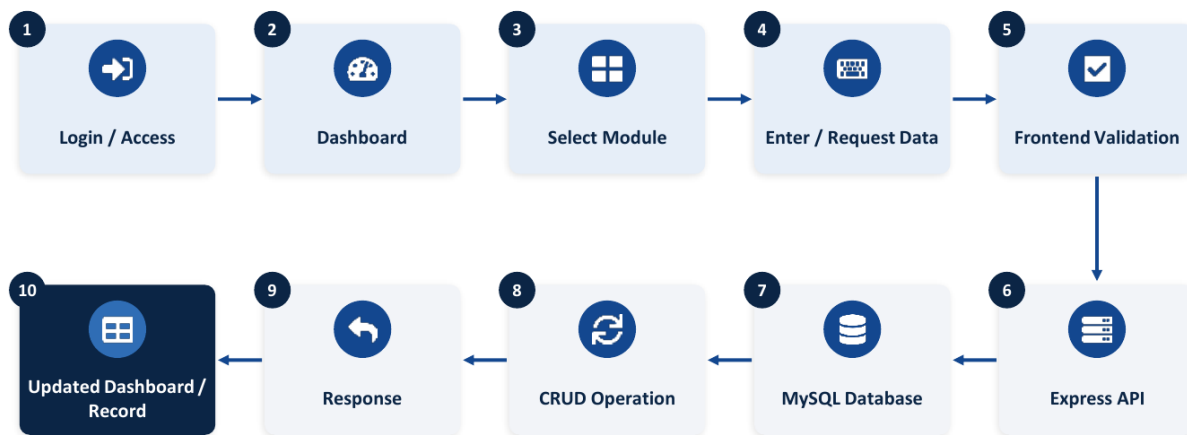


Figure 4.2 – Methodology and request flow

4.2 Description of Each Layer (Frontend, Backend, Database)

Client

The client is the person using the system, working in a web browser on a computer, tablet or phone. No software needs to be installed. The same URL serves all roles, and what each person can see and do is decided by the role they log in with.

Presentation Layer (Frontend)

The presentation layer consists of HTML pages styled with CSS and made interactive with JavaScript. It displays data in cards and tables, collects input through forms, performs first-level validation (required fields, correct e-mail format) and sends requests to the backend. It never talks to the database directly, so no database credentials are ever exposed to the browser.

Communication Layer (HTTP / REST API)

The frontend and backend communicate over HTTP using JSON. URLs identify resources (students, rooms, allocations, fees, complaints) and HTTP methods identify the action. Standard status codes (200, 201, 400, 401, 403, 404, 500) tell the frontend whether a request succeeded or why it failed.

Application Layer (Backend)

The application layer is the Node.js and Express server. It contains the business rules of the hostel: for example, an allocation is accepted only if the room exists, the student exists and the room's occupied count is less than its capacity. It also performs authentication, authorization, backend validation and error handling.

Data Layer (Database)

The data layer is the MySQL database. It stores persistent data in related tables and uses primary keys, foreign keys, NOT NULL and UNIQUE constraints to protect integrity. All modules share this single database, which is what removes duplication and keeps information consistent.

4.3 Deployment Architecture (Local, Cloud, CI/CD Pipelines)

Current deployment (local). The system runs on a single computer: MySQL runs as a local service, the Node.js server runs on a local port (for example 3000) and the frontend files are opened in a browser or served by Express as static files. This arrangement is sufficient for development, testing and the final demonstration, and could also be used on a computer in the hostel office on the local network.

Source management. The code is stored in a GitHub repository. Each team member works on a local clone and pushes changes to GitHub, which acts as the shared source of truth and as a backup.

Planned cloud deployment. In the future, the backend can be hosted on a cloud platform, the MySQL database on a managed database service and the static frontend on any static-hosting service. A CI/CD pipeline (for example GitHub Actions) could automatically run tests and redeploy whenever new code is pushed. Container-based deployment with Docker is also listed as future work in Chapter 11.

CHAPTER -5 DESIGN

5.1 ER Diagram

The Entity Relationship (ER) diagram below shows the main entities of the system and how they are related. A student is assigned rooms through allocations (one student can have many allocation records, because the allocation history is kept); each allocation refers to exactly one room, while a room can appear in many allocations; and one student can pay many fee records and raise many complaints.

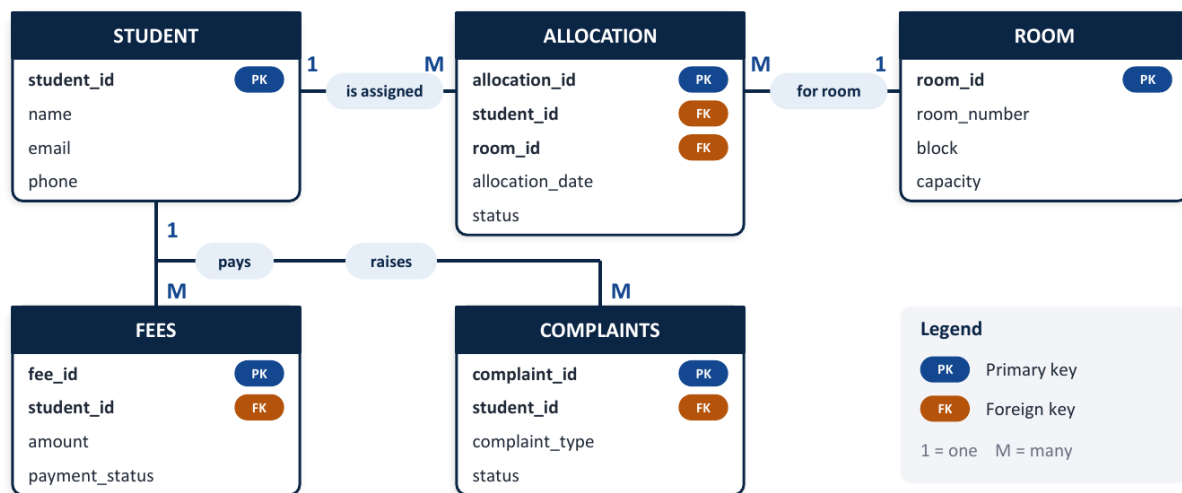


Figure 5.1 – Entity Relationship Diagram

Relationship	Cardinality	Meaning
STUDENT – ALLOCATION	1 : M	One student can have many allocation records (current and past).
ROOM – ALLOCATION	1 : M	One room can be allocated many times over time and to many students up to its capacity.
STUDENT – FEES	1 : M	One student can make many fee payments.
STUDENT – COMPLAINTS	1 : M	One student can raise many complaints.

STUDENT and ROOM are connected through ALLOCATION. The many-to-many relationship between students and rooms (over time) is resolved into two one-to-many relationships by the ALLOCATION table, which also stores the allocation date and status.

5.2 Database Schema

The relational schema consists of the following tables. PK denotes a primary key and FK denotes a foreign key.

Table	Columns
students	student_id (PK), name, email, phone, gender, course, year
rooms	room_id (PK), room_number, block, floor, capacity, occupied_count, status

Table	Columns
allocations	allocation_id (PK), student_id (FK → students), room_id (FK → rooms), allocation_date, status
fees	fee_id (PK), student_id (FK → students), amount, payment_date, payment_status
complaints	complaint_id (PK), student_id (FK → students), complaint_type, description, complaint_date, status

Sample SQL definitions

```
CREATE TABLE students (
  student_id VARCHAR(15) PRIMARY KEY,
  name       VARCHAR(100) NOT NULL,
  email      VARCHAR(100) UNIQUE NOT NULL,
  phone      VARCHAR(15),
  gender     VARCHAR(10),
  course     VARCHAR(50),
  year       INT
);

CREATE TABLE rooms (
  room_id      INT AUTO_INCREMENT PRIMARY KEY,
  room_number  VARCHAR(10) UNIQUE NOT NULL,
  block        VARCHAR(10),
  floor        INT,
  capacity     INT NOT NULL,
  occupied_count INT DEFAULT 0,
  status       VARCHAR(20) DEFAULT 'Available'
);

CREATE TABLE allocations (
  allocation_id VARCHAR(10) PRIMARY KEY,
  student_id    VARCHAR(15) NOT NULL,
  room_id       INT NOT NULL,
  allocation_date DATE NOT NULL,
  status        VARCHAR(20) DEFAULT 'Active',
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (room_id)    REFERENCES rooms(room_id)
);
```

The fees and complaints tables follow the same pattern, each holding a foreign key to students. Data types shown above are illustrative and may differ slightly in the final database script.

Normalisation

First normal form (1NF): every column holds a single atomic value and every table has a primary key. **Second normal form (2NF):** every non-key attribute depends on the whole primary key; for example, the room's block and capacity are stored once in the rooms table and not repeated in every allocation. **Third normal form (3NF):** no non-key attribute depends on another non-key attribute; student details are stored only in the students table and referenced by student_id everywhere else. As a

result there is no duplicated student or room information, updates need to be made in only one place, and the risk of contradictory data is removed.

Integrity rules

- Entity integrity: each table has a unique, non-null primary key.
- Referential integrity: foreign keys ensure that an allocation, fee or complaint always refers to an existing student (and an existing room, for allocations).
- Business rule: a room's `occupied_count` must never exceed its `capacity`; a room whose occupancy equals its capacity is marked as full.

CHAPTER -6 IMPLEMENTATION

Section 6.1 – Module-Wise Implementation

The system is implemented as a set of modules that share one MySQL database through the Express REST API. Each module has its own page(s) on the frontend and its own routes on the backend, but all of them use the same login, the same look and feel and the same database.

1. Student Management Module

This module maintains the master record of every hostel resident. Authorised users can add a student, view the list of students, update details, delete a record and search by name or student ID. The student ID (for example 2520090011) is the primary key and is used by every other module to link records to the student. Backend validation rejects duplicate IDs and malformed e-mail addresses.

2. Room Management Module

The room module maintains each room's number, block, floor, capacity, occupied count and status. Staff can add and update rooms and check availability. The occupied count is updated automatically when a room is allocated or vacated, so the list of available rooms is always current.

3. Room Allocation Module

This module connects students with rooms. An allocation record stores the allocation ID (such as A001), the student ID, the room number, the allocation date and the status. Before a new allocation is saved, the backend checks that the student exists, that the room exists and that the room still has space. Changing a room marks the old allocation as inactive and creates a new one, so the full allocation history is preserved. The Room Allocation page (Figure 14.3) shows summary cards – Total Allocations, Active Allocations and Other Records – and a searchable table of allocation records.

4. Fee Management Module

The fee module records each payment against a student with the amount, payment date and payment status, and allows users to view and search payment records. Because every fee row carries the student ID as a foreign key, the fee status of any student can be retrieved instantly.

5. Complaint Management Module

Students or staff can register a complaint with a type, a description and a date. Each complaint carries a status (for example Pending, In Progress or Resolved) which authorised staff update as the work progresses. This replaces verbal complaints and loose paper notes with a trackable record.

6. Authentication and Authorization Module

This module controls who can enter the system and what they can do. Users sign in with an e-mail address and password on the login page, and every protected request is checked for a valid login and for the role needed. It is described in detail in Section 6.4.

7. Admin Dashboard Module

The dashboard is the central place to reach and manage all modules. After login it greets the user ("Welcome, Admin"), shows a Hostel Overview with live counts of Total Students, Total Rooms,

Active Allocations and Complaints (Figure 14.2), and offers an Admin Control Center with links to each module.

8. Reports Module

The reports module gives summaries of students, rooms, fees and complaints. These summaries are produced by SQL queries that count and group data, for example the number of occupied versus available rooms or the number of pending fees.

Collectively, the modular implementation ensures that each part of the system has a clear responsibility while still sharing a single, consistent data store. New modules, such as notifications or attendance, can be added later without redesigning the existing ones.

Section 6.2 – Front-End Logic (UI Rendering and State Management)

The frontend is implemented as a set of HTML pages that share one stylesheet and use JavaScript to talk to the REST API. Its design aims to translate the backend's data into a simple, clear interface.

Page structure

Every page follows the same structure. A header with the navigation bar (logo, Home, Features, About and a Login button on the public pages; the user's role and a Logout button on internal pages) is followed by the main content. On the home page the content is a hero section with the system title, a short description, Get Started and Explore Features buttons, and illustrated cards for Students, Rooms and Fees. The login page uses a login-container with two sections: a left introduction ("Manage your hostel smarter" with the points Student Management, Room Allocation and Fees & Complaints) and a right-hand login card with Email Address and Password fields inside a form with the id loginForm. Decorative background orbs give the page its soft gradient look.

The HTML source of these pages is shown in Appendix Figures 14.4 to 14.8, and the rendered home page is shown in Figure 14.1.

UI rendering

Tables and dashboard counters are rendered from JSON data. After a page loads, JavaScript calls the relevant API endpoint with fetch, receives an array of records, and builds table rows or updates counter values in the page. When a record is added, updated or deleted, the page requests the data again so that the interface always reflects what is in the database.

```
async function loadAllocations() {
  const res = await fetch('/api/allocations');
  const data = await res.json();
  const body = document.getElementById('allocationBody');
  body.innerHTML = data.map(a => `
    <tr><td>${a.allocation_id}</td><td>${a.student_id}</td>
      <td>${a.room_number}</td><td>${a.allocation_date}</td>
      <td><span class="badge">${a.status}</span></td></tr>`).join('');
}
```

The code above is an illustration of the approach used to render table data.

State management

State on the frontend is deliberately kept simple. The authoritative state of the hostel always lives in the database; the browser keeps only short-lived state such as the logged-in user's role, the current search text and the form values being edited. After each successful action the page re-fetches the data, which avoids the browser and the database drifting out of step.

Validation and feedback

Forms use HTML attributes such as `required` and `type="email"` plus JavaScript checks to catch missing or wrong input before it is sent. The login form, for example, requires both the e-mail and the password and offers a button to show or hide the password. Status values such as `Active` are displayed as coloured badges so that they can be recognised at a glance, and a search box above the allocation table filters rows as the user types.

Responsiveness

The pages use a viewport meta tag and flexible CSS layouts so that cards and tables adapt to different screen widths.

6.3 API Integration

API integration is the link between the frontend pages and the backend logic. All data exchanged between them uses JSON over HTTP. The table below lists the REST endpoints designed for the system and what each one does.

Method	Endpoint	Purpose
GET	/api/students	List all students
GET	/api/students/:id	Get one student by ID
POST	/api/students	Add a new student
PUT	/api/students/:id	Update a student
DELETE	/api/students/:id	Delete a student
GET	/api/rooms	List rooms with occupancy and availability
POST	/api/rooms	Add a room
PUT	/api/rooms/:id	Update room details or status
POST	/api/allocations	Assign a student to a room
GET	/api/allocations	List allocation records
PUT	/api/allocations/:id	Change an allocation or its status
POST	/api/fees	Record a fee payment
GET	/api/fees	List payments and payment status
POST	/api/complaints	Register a complaint
GET	/api/complaints	List complaints

Method	Endpoint	Purpose
PUT	/api/complaints/:id	Update complaint status

A typical request and the matching Express route are shown below. The backend validates the input, runs a parameterised query and returns JSON.

```
// Frontend
fetch('/api/students', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify({ student_id, name, email, phone })
});

// Backend (Express)
router.post('/students', async (req, res) => {
  const { student_id, name, email, phone } = req.body;
  if (!student_id || !name || !email)
    return res.status(400).json({ error: 'Missing fields' });
  await db.query(
    'INSERT INTO students (student_id, name, email, phone) VALUES (?, ?, ?, ?)',
    [student_id, name, email, phone]);
  res.status(201).json({ message: 'Student added' });
});
```

The placeholders (?) in the SQL statement are filled in by the database driver, which is what protects the system against SQL injection. Errors are caught and converted into clear JSON messages with an appropriate status code so that the frontend can show a meaningful message to the user.

6.4 Authentication and Authorisation

Authentication answers the question “Who are you?”. The user enters an e-mail address and password on the login page. The backend looks up the account, compares the entered password with the stored hash and, if they match, starts a login session for that user and sends back the user’s role. Passwords are never stored in plain text; only a salted hash is saved, so that even someone who sees the database cannot read the passwords.

Authorisation answers the question “What are you allowed to do?”. The system uses role-based access control. Every protected route checks that the request comes from a logged-in user and that the user’s role is allowed to perform the action. A request from an unauthenticated user is rejected with status 401, and a request from a user whose role lacks permission is rejected with status 403. Checks are made on the server, not only in the browser, so hiding a button in the interface is never the only protection.

Role	Typical permissions
Admin	Full system access: all modules, reports, user management and access control.
Warden	Manage students, rooms, allocations and complaints; view reports.

Role	Typical permissions
Staff	Update rooms, record fee payments and update complaint status.
Student	View own details, allocation and fee status; register and track own complaints.

After sign-in the user is taken to the page for their role; for example, an administrator sees the “Welcome, Admin – Administrator, Full System Access” dashboard shown in Figure 14.2.

CHAPTER -7 FEATURES

7.1 List of Main Features

The Smart Hostel Management System offers the following main features:

No.	Feature	Brief description
1	Secure login	E-mail and password sign-in with role-based access for Admin, Warden, Staff and Student.
2	Admin dashboard	Welcome page with live counts of students, rooms, active allocations and complaints, and links to every module.
3	Student management	Add, view, update, delete and search student records.
4	Room management	Maintain room details, track occupancy and check availability.
5	Room allocation	Assign or change rooms and keep the allocation history, with a searchable list of records.
6	Fee management	Record payments, view payment status and search payment records.
7	Complaint management	Register complaints and update their status.
8	Reports	Summaries of students, rooms, fees and complaints.
9	Search and filter	Quickly find records in tables using the search box.
10	Validation and security	Frontend and backend validation, parameterised queries and hashed passwords.

7.2 How Each Feature Works

1. Secure login

User perspective: the user opens the login page, enters an e-mail address and password and clicks the sign-in button. The page offers a button to show the password and a “Back to Home” link. On success, the user lands on the page for their role; on failure, a clear message is shown. **Technical description:** the form (id loginForm) is validated in the browser and the details are sent to the backend. The server finds the account, compares the password hash and, if correct, starts a session and returns the role. Protected routes check the session and the role on every request.

2. Admin dashboard

User perspective: after login the administrator sees “Welcome, Admin” and a Hostel Overview with four cards – Total Students, Total Rooms, Active Allocations and Complaints – followed by an Admin Control Center with links to each module. **Technical description:** the page requests summary data from the API and writes the counts into the cards. Because the counts come from database queries such as COUNT(*), they always reflect the current data.

3. Student management

User perspective: the user opens the Students page, sees a table of students, and can add a new student through a form, edit or delete an existing one, or search by name or ID. **Technical**

description: the page calls GET /api/students to list, POST to add, PUT to update and DELETE to remove. The backend validates fields, uses parameterised SQL and returns a JSON message. A student with existing allocations, fees or complaints cannot be removed without handling the dependent records, because the foreign keys protect integrity.

4. Room management

User perspective: the user views all rooms with block, floor, capacity, occupied count and status, adds new rooms and edits details. **Technical description:** rooms are served by GET/POST/PUT endpoints. The status field (for example Available or Full) is derived from occupied_count compared with capacity, so availability checks are a simple query.

5. Room allocation

User perspective: the user chooses a student and a room and saves the allocation; the Room Allocation page then lists records such as A001 (student 2520090011, room 202, 20 Sept 2026, Active) with summary cards and a search box (Figure 14.3). To move a student, the user changes the allocation. **Technical description:** POST /api/allocations checks that the student and room exist and that occupied_count is below capacity, inserts the allocation, and increments occupied_count. When a room is changed, the old allocation's status is updated, the old room's count is decreased and a new allocation is created, so that history is kept. These steps are performed as one database transaction so that a failure cannot leave the counts inconsistent.

6. Fee management

User perspective: the user records a payment against a student with the amount and date, and can view or search payments and their status. **Technical description:** POST /api/fees inserts a row with the student_id foreign key, amount, payment_date and payment_status; GET /api/fees returns the list, optionally joined with student names.

7. Complaint management

User perspective: a complaint is registered with a type and description; staff later change its status as it is handled; the list shows all complaints. **Technical description:** POST /api/complaints inserts a new row with the current date and an initial status; PUT /api/complaints/:id updates the status. The dashboard's Complaints counter is derived from this table.

8. Reports

User perspective: the administrator opens the reports page for summaries of students, rooms, fees and complaints. **Technical description:** reports are produced by aggregate SQL queries (COUNT, SUM, GROUP BY) over the same tables, so no data is duplicated for reporting.

9. Search and filter

User perspective: typing into the search box above a table (for example "Search allocations...") narrows the list immediately. **Technical description:** the filter is implemented in JavaScript over the loaded rows, or by passing the search text to the API where the table is large.

10. Validation and security

User perspective: wrong or incomplete input is rejected with an understandable message, and users cannot open pages meant for other roles. **Technical description:** HTML attributes and JavaScript validate input in the browser; the backend validates again because the browser cannot be trusted; SQL statements are parameterised; passwords are hashed; and each route checks login and role.

CHAPTER -8 TESTING

Testing was planned to verify that every module works as specified, that the frontend, backend and database cooperate correctly and that invalid input and unauthorised access are handled safely. Testing combined manual checks of the user interface with API tests made in Postman and direct checks of the database.

8.1 Postman Testing (Frontend/Backend)

Postman was used to test the REST API independently of the user interface. By sending HTTP requests directly to the endpoints, each backend function could be checked in isolation, and the status code and JSON body of every response could be inspected. A Postman collection was organised by module (Students, Rooms, Allocations, Fees, Complaints and Authentication). A variable for the base URL (for example `http://localhost:3000`) was used so that the same requests can be run on any machine.

Module	Request	What is verified
Students	POST /api/students	Valid body returns 201; missing fields return 400; duplicate ID is rejected.
Students	GET /api/students and /api/students/:id	Returns the list and a single record; unknown ID returns 404.
Students	PUT and DELETE /api/students/:id	Record is updated or removed; response confirms the change.
Rooms	GET/POST/PUT /api/rooms	Rooms are listed, created and updated with correct occupancy fields.
Allocations	POST /api/allocations	Allocation is created for a valid student and room; full room is rejected.
Fees	POST/GET /api/fees	Payment is stored against the student and listed with its status.
Complaints	POST/GET/PUT /api/complaints	Complaint is registered, listed and its status updated.
Authentication	POST login	Correct credentials succeed; wrong credentials return 401.

For the frontend, the browser's developer tools (Network and Console tabs) were used to confirm that each page sends the correct request and renders the response properly. Screenshots of the Postman collection and sample responses can be added in Appendix 14 once the final test run is recorded.

8.2 Integration Testing

Integration testing checks that the modules work correctly together, following the same path that a real user takes: login → dashboard → select module → enter or request data → frontend validation →

Express API → MySQL → CRUD operation → response → updated dashboard or record. The following scenarios were used:

1. Add a student, then allocate that student to a room, and confirm that the room's occupied count increases and the allocation appears in the Room Allocation list.
2. Try to allocate a student to a room that is full and confirm that the system refuses.
3. Record a fee payment for a student and confirm that it appears with the correct student in the fees list.
4. Register a complaint, change its status and confirm that the dashboard's complaint count and the list agree.
5. Log in with different roles and confirm that each role sees only the pages and actions allowed for it.

The dashboard (Figure 14.2) and the Room Allocation page (Figure 14.3) provide visible evidence of the integration: the summary counts on the dashboard are consistent with the records shown in the module pages.

8.3 Tools Used for Testing

Tool	Use
Postman	Sending requests to REST endpoints and checking status codes and JSON responses.
Browser developer tools	Inspecting network calls, console errors and responsive layouts.
MySQL Workbench	Running SELECT queries to verify that data was stored correctly.
Web browsers (Chrome, Edge, Firefox)	Checking appearance and behaviour across browsers.
VS Code and Node.js terminal	Running the server and reading logs and error messages.

8.4 Test Cases and Results

The table below lists the main test cases. Cases marked “Pass” are confirmed by the screenshots included in Appendix 14. For the remaining cases the expected result is listed and the actual outcome should be recorded from the final Postman run.

ID	Test case	Expected result	Status
TC01	Open the home page	Home page shows title, Get Started and Explore Features buttons and navigation bar.	Pass (Fig. 14.1)
TC02	Open the login page	Login card with Email and Password fields is displayed.	Pass (Fig. 14.4–14.8)
TC03	Admin login	Admin dashboard with welcome message and module links opens.	Pass (Fig. 14.2)

ID	Test case	Expected result	Status
TC04	Dashboard summary counts	Counters show total students, total rooms, active allocations and complaints from the database.	Pass (Fig. 14.2)
TC05	View allocation records	Allocation table lists ID, student ID, room number, date and status.	Pass (Fig. 14.3)
TC06	Login with wrong password	Request rejected with an error message; no session created.	Record after run
TC07	Add student with valid data	Record saved; status 201; student appears in list.	Record after run
TC08	Add student with missing fields	Status 400 with an error message; nothing saved.	Record after run
TC09	Update and delete a student	Changes are stored; record removed on delete.	Record after run
TC10	Allocate student to available room	Allocation created; occupied count increases by one.	Record after run
TC11	Allocate student to a full room	Request refused; no allocation created.	Record after run
TC12	Record fee payment	Payment saved against the student with correct status.	Record after run
TC13	Register and update a complaint	Complaint stored; status change saved and visible.	Record after run
TC14	Access admin page as Student	Access denied (401/403); page not shown.	Record after run
TC15	SQL injection attempt in a text field	Input treated as plain data; no query is altered.	Record after run

The results confirm that the visible user-interface flows work and that the dashboard counts are consistent with the stored allocation records, while the API and security cases are defined so that they can be re-run after every change as regression checks.

CHAPTER -9 DEPLOYMENT

9.1 Steps to Deploy

The following steps deploy the system on a local computer or on a server:

1. **Install the prerequisites:** Node.js (LTS), npm and MySQL Server.
2. **Get the project:** clone the GitHub repository or copy the project folder (with the frontend and backend folders) to the computer.
3. **Create the database:** open MySQL Workbench or the MySQL command line and run the SQL script that creates the database and the tables students, rooms, allocations, fees and complaints (and the users table).
4. **Configure the environment:** create a .env file in the backend folder with the database host, user, password and database name, and the server port (see Section 9.2).
5. **Install the dependencies:** open a terminal in the backend folder and run npm install.
6. **Start the server:** run node server.js (or npm start). The terminal should show that the server is listening and that the database connection succeeded.
7. **Open the application:** in a browser open the frontend home page (index.html), or the server URL if Express serves the frontend, and sign in.
8. **Verify:** open the dashboard, add a test record and check it in MySQL Workbench.

```
git clone <repository-url>
cd backend
npm install
node server.js
```

9.2 Environment Configuration

Settings that change from machine to machine – and secrets that must not be written into the source code – are kept in a .env file that is read when the server starts. This file is excluded from version control (listed in .gitignore) so that passwords are never pushed to GitHub.

Variable	Meaning	Example
PORT	Port on which the Express server listens	3000
DB_HOST	Host name of the MySQL server	localhost
DB_USER	MySQL user name	root
DB_PASSWORD	MySQL password	(kept private)
DB_NAME	Name of the hostel database	smart_hostel
SESSION_SECRET / JWT_SECRET	Secret used to sign login sessions or tokens	(kept private)

The variable names above are given as the recommended configuration; the exact names should match those used in the project's configuration file.

9.3 Hosting of Frontend and Backend

Local hosting. For development and demonstration, the backend runs on the developer's computer and the frontend is opened in the browser. This needs no cost and no internet connection.

Institutional hosting. The application can be run on a computer in the hostel office or on a college server so that the warden and staff can use it over the campus network. MySQL and Node.js are installed on that machine and the steps in Section 9.1 are followed.

Cloud hosting (planned). The frontend files are static and can be placed on any static hosting service; the Node.js backend can run on a cloud application platform; and MySQL can be run as a managed database. In that case the API URL in the frontend is changed to the public address of the backend, HTTPS is enabled and the allowed origins (CORS) are restricted to the frontend's address.

CHAPTER -10 CHALLENGES & LIMITATIONS

10.1 Issues Faced During Development

While designing and building the system, the team met a number of technical and practical challenges:

1. **Designing the database correctly:** deciding the tables, keys and relationships so that data is not repeated, especially the relation between students and rooms over time.
2. **Preventing room overbooking:** making sure that the number of students in a room never exceeds its capacity, even when changes are made quickly.
3. **Linking records:** connecting students, rooms, allocations, fees and complaints so that they always stay consistent.
4. **Wrong or incomplete input:** users may leave fields empty or enter data in the wrong format.
5. **Unauthorised access:** making sure that people can see and change only what their role permits.
6. **Security of passwords and queries:** avoiding plain-text passwords and SQL injection.
7. **Connecting frontend, backend and database:** getting requests, responses and data formats to match across the three layers, and handling errors such as failed database connections.
8. **Team coordination:** merging work from three team members and keeping the pages visually consistent.

10.2 Solutions Applied

Challenge	Solution applied
Data redundancy	Normalisation of the schema up to 3NF (1NF–3NF).
Linking records	Primary keys and foreign keys between all tables.
Room overbooking	Capacity and occupancy checks before each allocation, with updates in a single transaction.
Unauthorised access	Login and role-based access control checked on the server.
Wrong or incomplete input	Validation in the frontend and again in the backend.
SQL injection	Parameterised queries instead of building SQL from strings.
Storing passwords	Hashing instead of plain text.
Frontend–backend integration	Clear REST API design, JSON responses and testing with Postman.
Team coordination	Git and GitHub for version control; a shared stylesheet for consistent design.

10.3 Current Limitations or Known Bugs

The present version is a functional academic project, and the following limitations are acknowledged:

- Fee handling records payments but does not connect to an online payment gateway.
- There are no e-mail or SMS notifications; complaint and fee updates must be checked in the system.
- Reports are summary-level; there is no advanced analytics dashboard or automatic report generation.
- The system is designed for a single hostel and has not been load-tested for very large numbers of simultaneous users.
- It is a web application only; there is no dedicated mobile app.
- It currently runs locally; cloud deployment, automatic backups and CI/CD are not yet in place.
- Complete automated test coverage is not yet available; most testing is manual and through Postman.

CHAPTER -11 FUTURE ENHANCEMENTS

11.1 Planned Features

The foundation built in this project can be extended in many useful ways:

Planned feature	Benefit
Mobile application	Allows students and wardens to use the system from their phones.
Online fee payment integration	Students can pay hostel fees online and receipts are recorded automatically.
E-mail / SMS notifications	Automatic alerts for fee due dates, room allocation and complaint status changes.
QR-based hostel attendance	Quick and accurate attendance and entry/exit tracking.
Advanced analytics dashboard	Charts and trends of occupancy, fee collection and complaints.
Automated reports	Scheduled reports generated and exported without manual effort.
Role-based access expansion	More detailed roles and permissions.
Cloud deployment	Access from anywhere with managed hosting and backups.

11.2 Possible Integrations or Optimisations

- **MongoDB / NoSQL integration** where appropriate, for example for logs or notifications that do not fit a rigid table.
- **Microservice-based architecture** so that modules such as fees or complaints can be scaled and deployed independently.
- **Docker containerisation** to package the backend and database so that the system runs identically on any machine.
- **CI/CD pipelines** (for example GitHub Actions) to run automated tests and deploy on every push.
- **Performance optimisation** through database indexes on frequently searched columns, pagination of long lists and caching of dashboard counts.
- **Security hardening** with HTTPS, rate limiting on login, audit logs of administrative actions and regular backups.

These items are possible future extensions and are not features of the current version.

CHAPTER -12 CONCLUSION

12.1 Summary of the Project

The Smart Hostel Management System was conceived to solve a common and practical problem: the slow, error-prone, paper-based management of hostel records. The project replaces scattered registers with a centralised web application whose data is held in a single, normalised MySQL database and accessed through a Node.js and Express REST API and a clean HTML, CSS and JavaScript interface.

The system brings students, rooms, allocations, fees and complaints together, links them through primary and foreign keys, and gives authorised users a simple way to add, view, update, delete and search information. An admin dashboard provides an instant overview of the hostel, and role-based access keeps the data safe.

12.2 What Was Achieved

- A centralised database design with five core related tables, normalised to 3NF.
- A web interface with a home page, login page, admin dashboard and module pages such as Room Allocation, with a consistent and modern design.
- Modules for student, room, allocation, fee and complaint management with CRUD operations and search.
- A REST API design covering all modules, using standard HTTP methods and JSON.
- Role-based authentication and authorization for Admin, Warden, Staff and Student users.
- Protection against common problems: room overbooking, SQL injection, plain-text passwords and invalid input.
- A well-organised code base managed with Git and GitHub, ready for further extension.

12.3 Skills Learned During Development

Area	Skills gained
Database engineering	Requirement analysis, ER modelling, schema design, normalisation, writing SQL with constraints and joins.
Backend development	Building REST APIs with Node.js and Express, middleware, validation and error handling.
Frontend development	Semantic HTML, responsive CSS, JavaScript with fetch, building forms, tables and dashboards.
Security	Authentication, role-based authorization, password hashing, parameterised queries.
Tools and process	Git and GitHub, VS Code, MySQL Workbench, Postman and technical documentation.
Teamwork	Dividing modules, coordinating through version control, reviewing each other's work and presenting the project.

In conclusion, the project met its aim of providing a simple, secure and efficient solution for managing day-to-day hostel operations, and it gives the team a strong foundation in database-backed web development.

CHAPTER -13 REFERENCES

Books

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson Education.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill Education.

Pressman, R. S., & Maxim, B. R. (2019). Software Engineering: A Practitioner's Approach (8th ed.). McGraw-Hill Education.

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education.

Haverbeke, M. (2018). Eloquent JavaScript (3rd ed.). No Starch Press.

Course and Project Documents

Course Handout – Database Systems Engineering and Distributed Backend Development (25CS1302E), K L University.

Project Abstract Submission Form – Smart Hostel Management System, Team 2, Section 13.

Documentation and Tutorials

MySQL Reference Manual. Retrieved from <https://dev.mysql.com/doc>

Node.js Documentation. Retrieved from <https://nodejs.org/docs>

Express.js Documentation. Retrieved from <https://expressjs.com>

MDN Web Docs – HTML, CSS and JavaScript. Retrieved from <https://developer.mozilla.org>

Git Documentation. Retrieved from <https://git-scm.com/doc>

GitHub Docs. Retrieved from <https://docs.github.com>

Postman Learning Center. Retrieved from <https://learning.postman.com>

Research

Fielding, R. T. (2000). Architectural Styles and the Design of Network-based Software Architectures (Doctoral dissertation). University of California, Irvine.

CHAPTER -14 APPENDICES

Screenshots of the app

The following screenshots show the Smart Hostel Management System in operation.

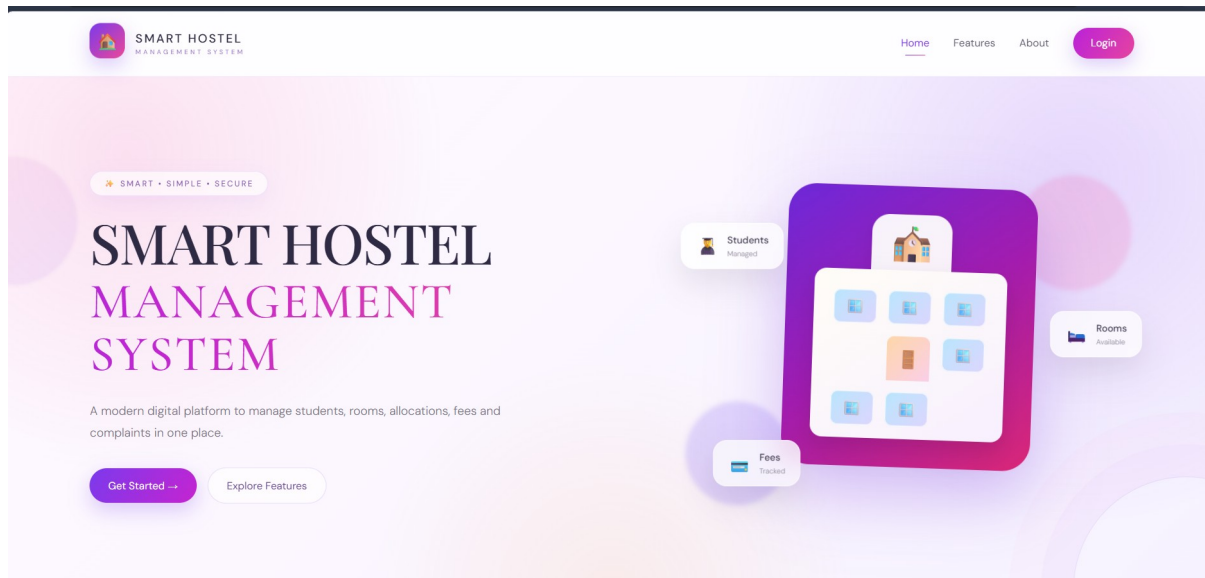


Figure 14.1 – Home page of the Smart Hostel Management System

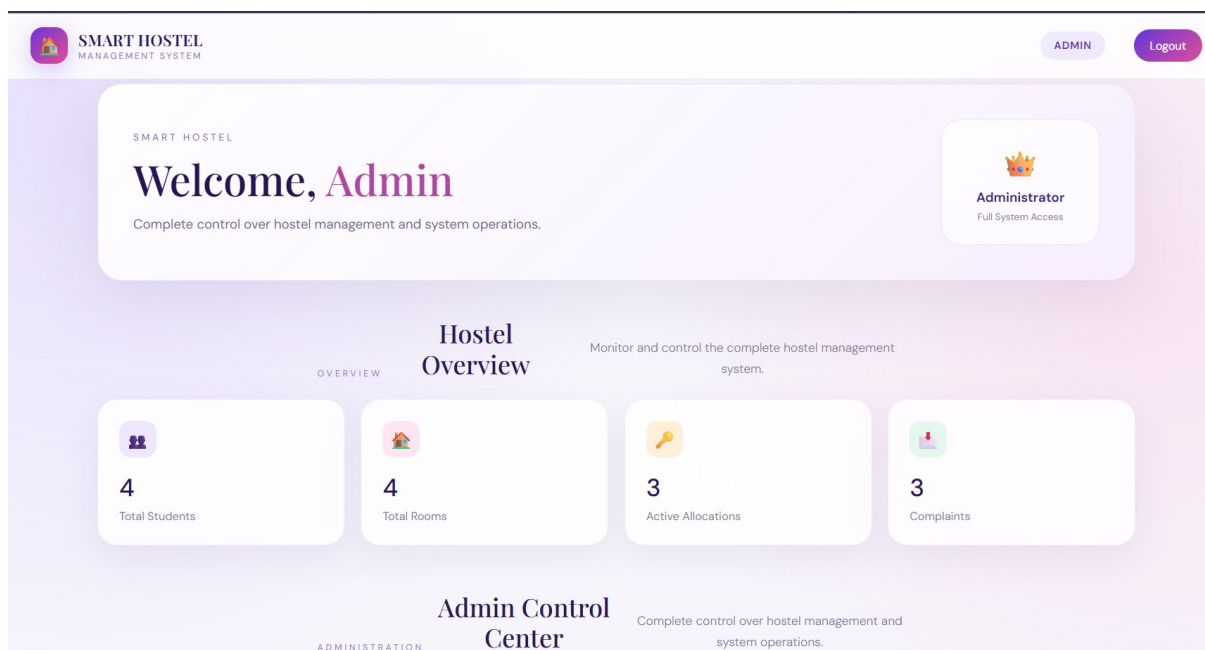


Figure 14.2 – Admin dashboard (Hostel Overview)

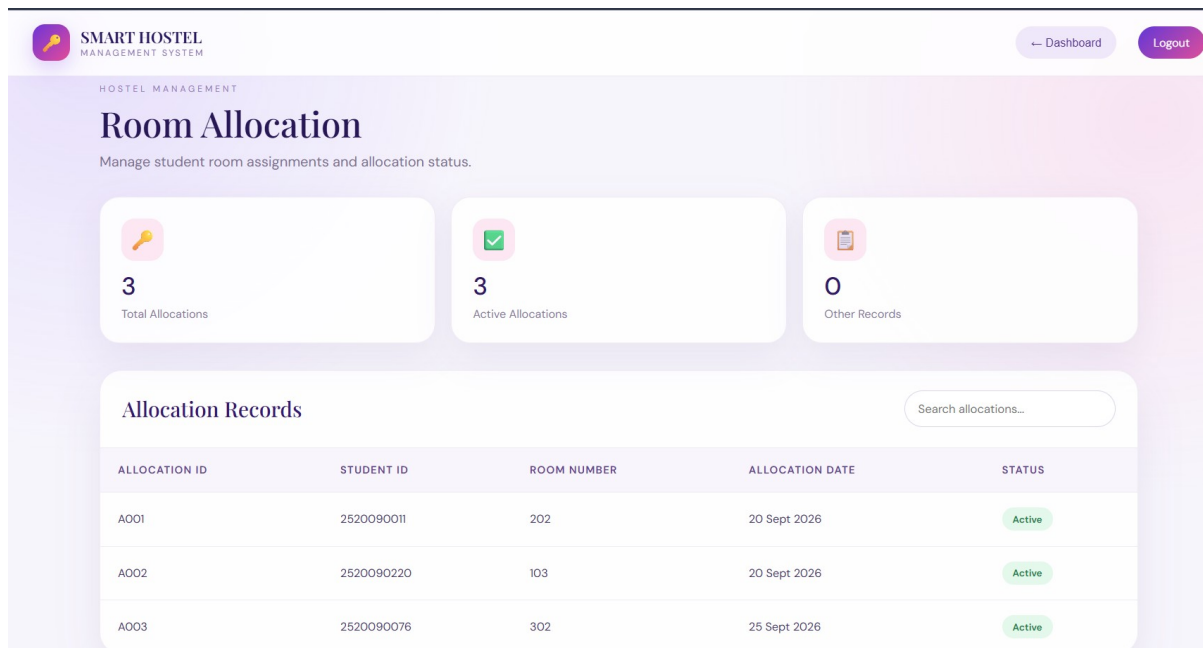


Figure 14.3 – Room Allocation page with allocation records

Sample code snippets

The screenshots below show parts of the frontend source code in VS Code.

index.html – document head, navigation bar and start of the hero section:

```
frontend > index.html > html > body > header.navbar > nav > a.active
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6
7   <title>Smart Hostel Management System</title>
8
9   <link rel="stylesheet" href="css/style.css">
10 </head>
11
12 <body>
13
14   <!-- NAVBAR -->
15   <header class="navbar">
16
17     <div class="logo">
18       <div class="logo-icon">🏠</div>
19       <div>
20         <h2>SMART HOSTEL</h2>
21         <span>MANAGEMENT SYSTEM</span>
22       </div>
23     </div>
24
25     <nav>
26       <a href="index.html" class="active">Home</a>
27       <a href="#features">Features</a>
28       <a href="#about">About</a>
29       <a href="login.html" class="login-btn">Login</a>
30     </nav>
31
32   </header>
33
34
35   <!-- HERO SECTION -->
36   <section class="hero">
```

Figure 14.4 – index.html (head, navbar and hero section)

login.html – page head, background and login container:

```
frontend > <> login.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7
8      <title>Login | Smart Hostel Management System</title>
9
10     <link rel="stylesheet" href="css/style.css">
11 </head>
12
13 <body class="login-page">
14
15     <div class="login-background">
16         <div class="login-orb orb-one"></div>
17         <div class="login-orb orb-two"></div>
18         <div class="login-orb orb-three"></div>
19     </div>
20
21
22     <!-- LOGIN CONTAINER -->
23
24     <main class="login-container">
25
26         <!-- LEFT SIDE -->
27
28         <section class="login-intro">
29
30             <a href="index.html" class="back-home">
31                 ← Back to Home
32             </a>
33
34             <div class="login-brand">
35                 <div class="login-logo">🏠</div>
36             </div>
37         </section>
38     </main>
39 </body>
40 </html>
```

Figure 14.5 – login.html (head, background orbs and login container)

login.html – introduction text and feature list:

```
50         Manage your
51         <em>hostel smarter.</em>
52     </h1>
53
54     <p>
55         Access your hostel management dashboard
56         and manage everything from one place.
57     </p>
58
59 </div>
60
61
62 <div class="login-features">
63
64     <div>
65         <span>01</span>
66         <p>Student Management</p>
67     </div>
68
69     <div>
70         <span>02</span>
71         <p>Room Allocation</p>
72     </div>
73
74     <div>
75         <span>03</span>
76         <p>Fees & Complaints</p>
77     </div>
78
79 </div>
80
```

Figure 14.6 – login.html (heading and login features)

login.html – login card and form with the e-mail field:

```
83
84     <!-- LOGIN CARD -->
85
86     <section class="login-card">
87
88         <div class="login-card-heading">
89
90             <span>SECURE ACCESS</span>
91
92             <h2>Sign in</h2>
93
94             <p>
95                 Enter your details to continue
96             </p>
97
98         </div>
99
100
101         <form id="loginForm">
102
103             <!-- EMAIL -->
104
105             <div class="input-group">
106
107                 <label for="email">
108                     Email Address
109                 </label>
110
111                 <div class="input-wrapper">
112
113                     <span>✉</span>
114
115                     <input
```

Figure 14.7 – login.html (login card and loginForm)

login.html – e-mail and password input groups:

```
117         id="email"
118         placeholder="Enter your email"
119         required
120     >
121
122 </div>
123
124 </div>
125
126
127 <!-- PASSWORD -->
128
129 <div class="input-group">
130
131     <label for="password">
132         Password
133     </label>
134
135     <div class="input-wrapper">
136
137         <span>=</span>
138
139         <input
140             type="password"
141             id="password"
142             placeholder="Enter your password"
143             required
144         >
145
146     <button
147         type="button"
```

Figure 14.8 – login.html (email and password inputs)

Installation/setup instructions

The Smart Hostel Management System runs on Node.js with a MySQL database. Setup takes a few steps:

1. Install Node.js (LTS version) and MySQL Server, and install Visual Studio Code and Git if they are not already present.
2. Download or clone the project folder from GitHub to your computer. If it is shared as a ZIP file, extract it first.
3. Open MySQL Workbench, create the database (for example smart_hostel) and run the provided SQL script to create the tables.
4. In the backend folder, create the .env file with your database details and port number.
5. Open a terminal in the backend folder and run npm install to install the dependencies.
6. Start the server with node server.js and wait for the message that the server is running and connected to MySQL.
7. Open index.html (or the server address, such as http://localhost:3000) in Google Chrome, Microsoft Edge or Firefox.

User manual or guide

Home page. When you open the system, the home page shows the navigation bar with Home, Features, About and Login. Click Login (or Get Started) to go to the sign-in page.

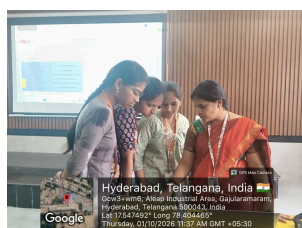
Signing in. Enter your registered e-mail address and password and click the sign-in button. Use the eye button to show or hide the password. Use “Back to Home” to return to the home page.

Dashboard. After signing in as Administrator you see “Welcome, Admin” and a Hostel Overview with the number of students, rooms, active allocations and complaints. Below it the Admin Control Center has a card for each module. Click a card to open that module.

Managing records. In each module page you can view the list of records, add a new record with the form, edit or delete an existing record, and use the search box to find a record quickly. On the Room Allocation page, the cards at the top show the number of total and active allocations, and the table below lists each allocation with its ID, student ID, room number, date and status.

Dashboard button and Logout. Use the Dashboard button at the top right of any module page to return to the dashboard, and click Logout when you have finished so that nobody else can use your session.

Geo Tag photos with guide



Review forms with guide signatures

[Insert scanned review forms with the guide’s signatures here]